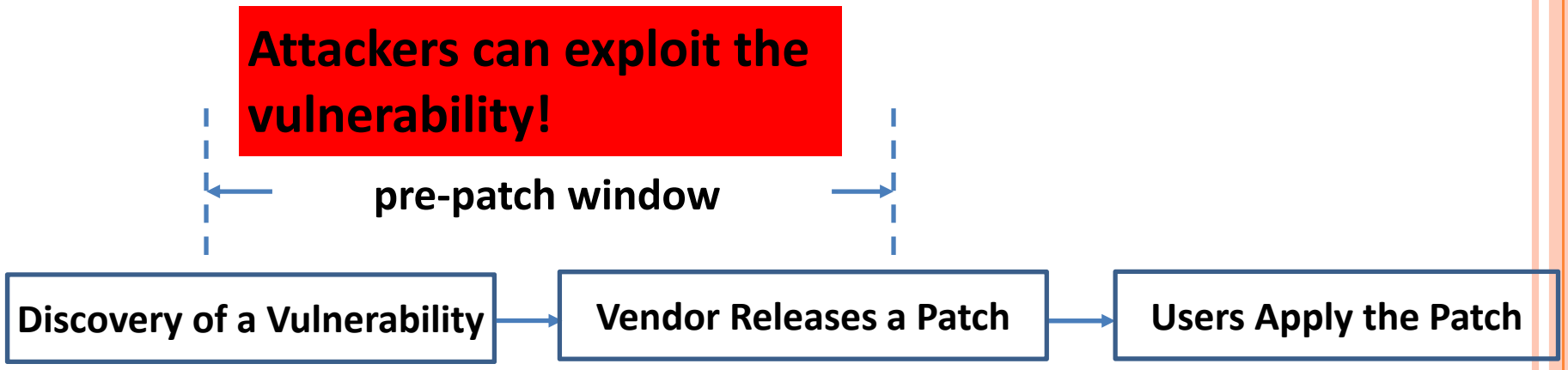
Pre-patch Window

- Attackers can leverage the window of time before a vulnerability is addressed.



Our Goal

- Automatically repair software vulnerabilities i.e. automated program repair
- Focuses on source code repair
  - Easier for developers to adopt

Pre-patch Window is Significant

- Study on 130 real-world vulnerabilities [1]
  - 7-30 days for 1/4 vulnerabilities
  - 30+ days for 1/3 vulnerabilities
  - 52 days on average

1. Z. Huang, M. D'Angelo, D. Miyani, D. Lie. *Talos: Neutralizing Vulnerabilities with Security Workaround for Rapid Response*. IEEE Symposium on Security & Privacy 2016.

Issues of Manual Repair

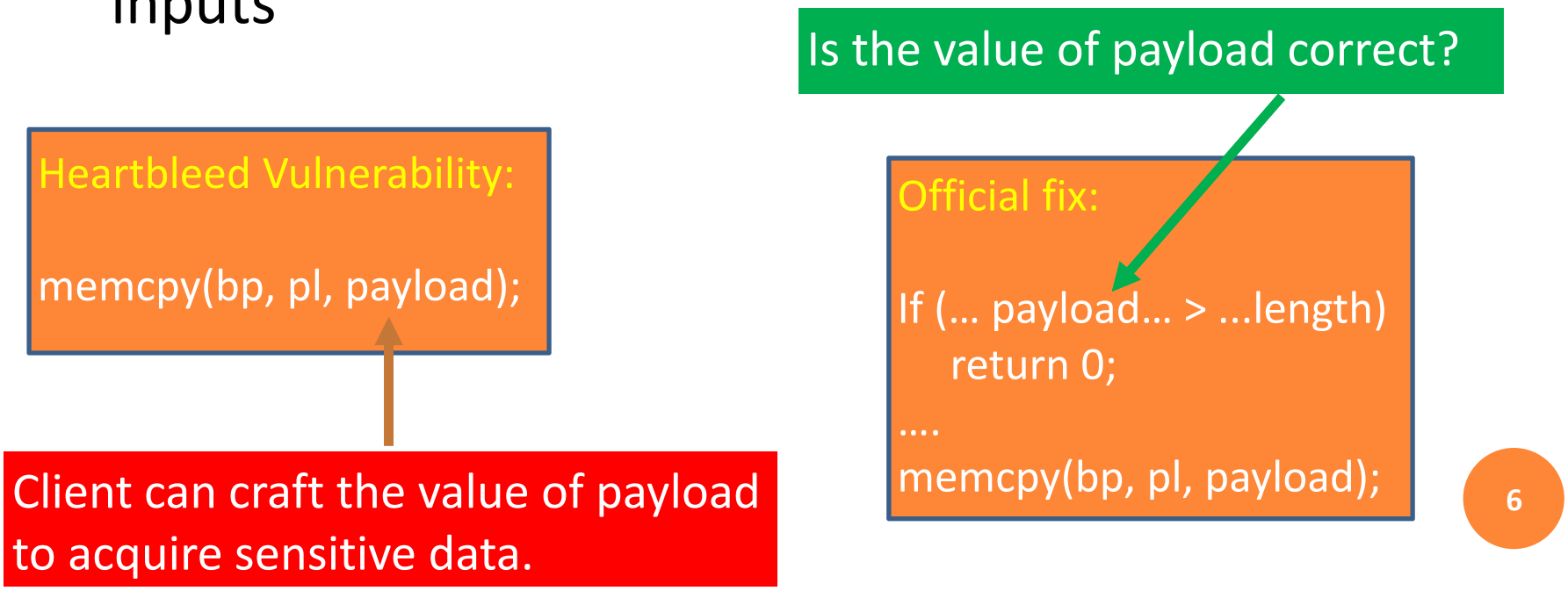
- Time required to construct a correct fix is significant.
  - It accounts for 89% of the time for releasing a patch.
- Con
  - Sc
    se

Multiple attempts of patching (Quotes from a bug report)

The developer: “This updates the previous patch...”
....
The developer: “This patch builds on the previous one...”
....
The developer: “I’ve just committed more changes...”
....
....
The tester: “I’m afraid I found a bug...”

How to Repair Vulnerabilities?

- Correcting vulnerable logic, e.g. race condition
- Preventing vulnerable code from being executed
- Adding checks to detect vulnerability-triggering inputs



Two Types of Repairs

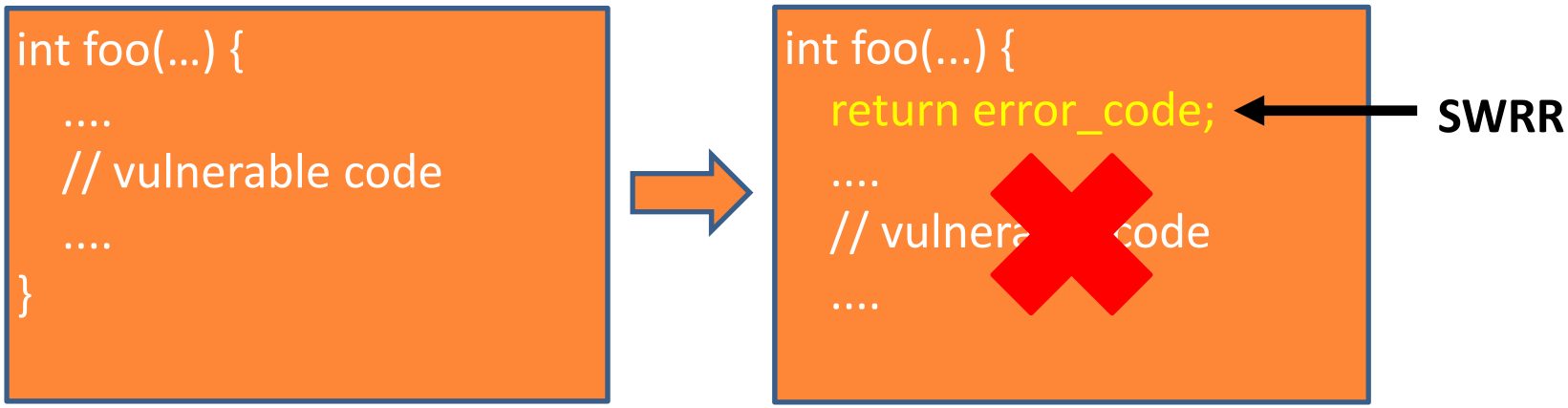
- Mitigation
  - Preventing vulnerabilities from being triggered
  - Rapid
- Fix
  - Removing vulnerabilities
  - Slow

MITIGATION

- Prevents execution of vulnerable code to thwarts exploits
  - Rapidly closes pre-patch window
- *Unobtrusiveness* is desirable
  - Only vulnerable code should be affected
- Trade off between functionality loss and security

Security Workaround for Rapid Response (SWRR)

- Designed to be simple and unobtrusive



- Oblivious to vulnerability types
- Requires minimum developer effort

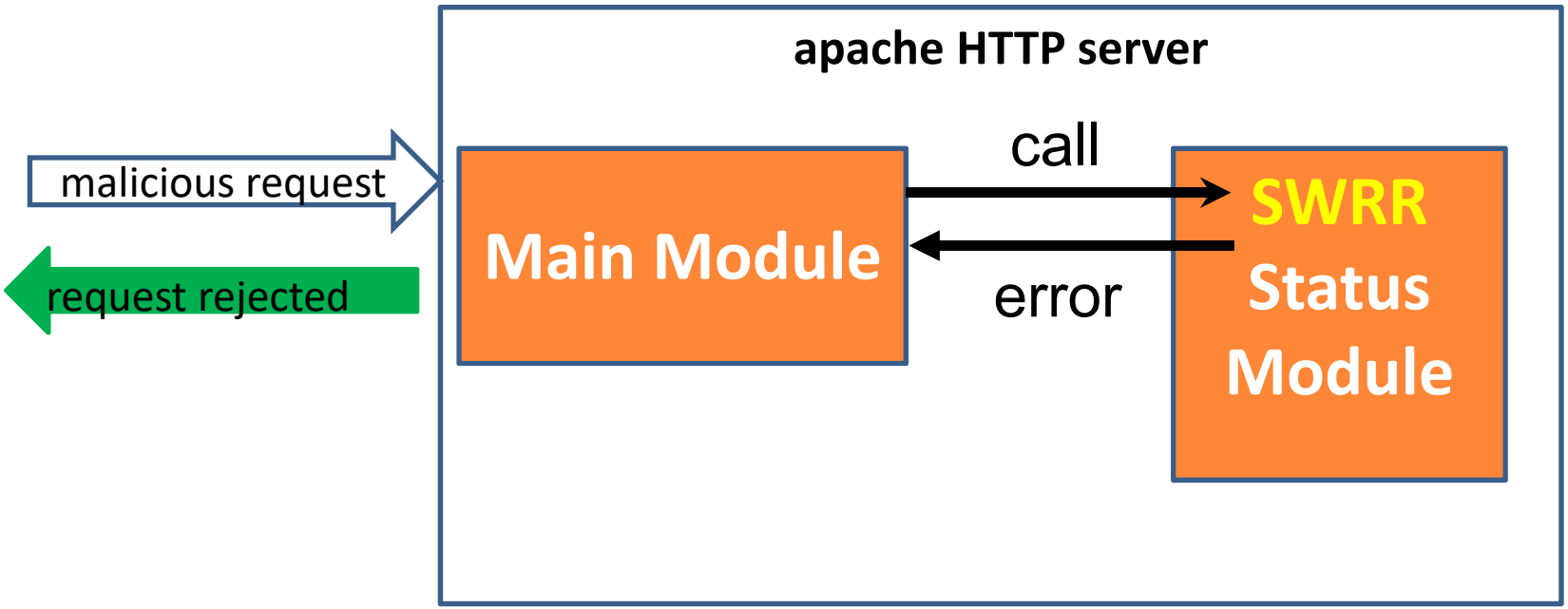
How to Achieve Unobtrusiveness?

- Terminate the target program?
- Throw an exception?
- Return to caller? What value to return?

```
int foo(...) {
   return ?;
   ....
   // vulnerable code
   ....
```

USING EXISTING ERROR RETURN VALUES

- Leveraging target program's own error handling mechanism



Identifying Error Return Values

- Documentation of common libraries or API functions
- Developers' annotations
- Observing behaviors of applications
- Analyzing error propagation
- Using heuristics

Analyzing Error Propagation

Upward Propagation

```
Int bar() {
   if (foo() == NULL)
      return -2;
   ....
```

foo: NULL

↓

bar: -2

Downward Propagation

```
Int bar() {
   ....
   if (spam() == -3)
      return -2;
```

bar: -2

↓

spam: -3

Direct Propagation

```
Int ham() {
   ....
   return bar();
   ....
```

bar: -2

↓

ham: -2

Using Heuristics

Error Logging

```
int baz() {
   ....
   If (error) {
         log_msg("ERROR!");
         return -1;
      }
```

Return NULL

```
char *foo() {
   ....
   if (error)
      return NULL;
   ....
```

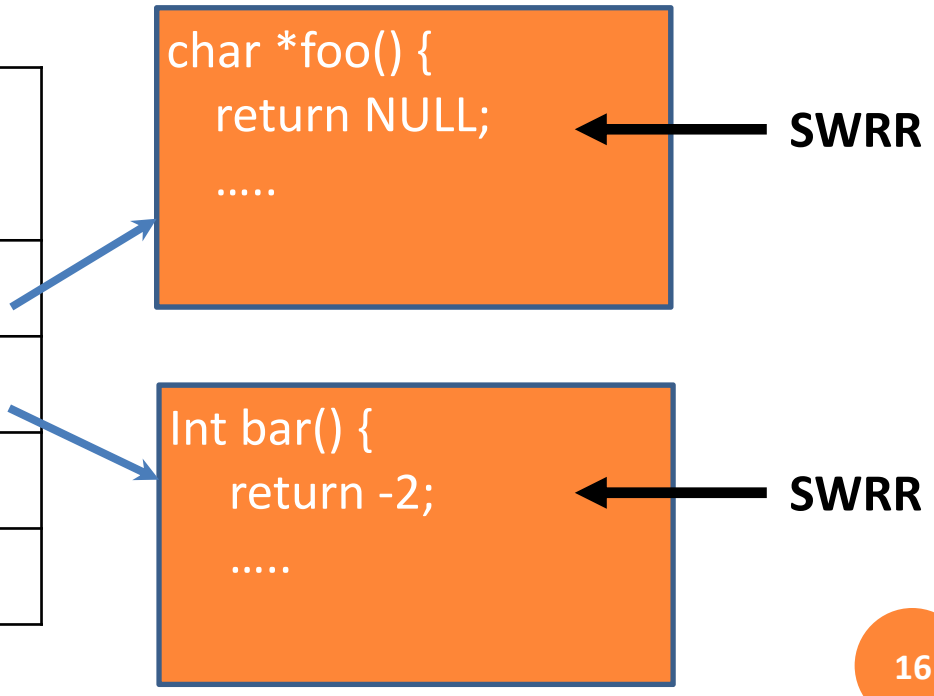
Combining Error Propagation Analysis and Heuristics

Function	Error Return Value
foo	NULL
bar	-2
spam	-3
ham	-2

GENERATING SWRRS

- An SWRR is simply a return statement:
  - return error;

Function	Error Return Value
foo	NULL
bar	-2
spam	-3
ham	-2



State-of-art Tools

- Talos
  - Generates source code SWRRs
  - Uses static program analysis
  - Instruments SWRRs into the source code of a target program

    https://github.com/huang-zhen/talos
- RVM
  - Generates binary code SWRRs
  - Instruments SWRRs into the binary of a target program

    https://gitlab.com/zhenhuang/RVM

TALOS DEMO – TARGET VULNERABILITY

CVE-2014-0226 Learn more at National Vulnerability Database (NVD)
• CVSS Severity Rating • Fix Information • Vulnerable Software Versions Mappings • CPE Information

Description

Race condition in the mod_status module in the Apache HTTP Server before 2.4.10 allows ren attackers to cause a denial of service (heap-based buffer overflow), or possibly obtain sensitiv credential information or execute arbitrary code, via a crafted request that triggers improper scoreboard handling within the status_handler function in modules/generators/mod_status.c

TALOS DEMO – GENERATING CFG & CDG

Talos generates CFG and CDG for *apache http server 2.4.7*

```
tools/.libs/gen_test_char.o@/home/james/code/httpd-2.4.7/srclib/apr/tools@gen_te
st_char.c@85@main_/home/james/code/httpd-2.4.7/srclib/apr/tools/gen_test_char.c@
85@@strchr@NULL,!=@-2@12
tools/.libs/gen_test_char.o@/home/james/code/httpd-2.4.7/srclib/apr/tools@gen_te
st_char.c@90@main_/home/james/code/httpd-2.4.7/srclib/apr/tools/gen_test_char.c@
90@@strchr@NULL,!=@-2@12
tools/.libs/gen_test_char.o@/home/james/code/httpd-2.4.7/srclib/apr/tools@gen_te
st_char.c@94@main_/home/james/code/httpd-2.4.7/srclib/apr/tools/gen_test_char.c@
94@@strchr@NULL,!=@-2@12
tools/.libs/gen_test_char.o@/home/james/code/httpd-2.4.7/srclib/apr/tools@gen_te
st_char.c@98@main_/home/james/code/httpd-2.4.7/srclib/apr/tools/gen_test_char.c@
98@@strchr@NULL,!=@-2@12
tools/.libs/gen_test_char.o@/home/james/code/httpd-2.4.7/srclib/apr/tools@gen_te
st_char.c@111@main_/home/james/code/httpd-2.4.7/srclib/apr/tools/gen_test_char.c
@111@@strchr@NULL,!=@-2@12
tools/.libs/gen_test_char.o@/home/james/code/httpd-2.4.7/srclib/apr/tools@gen_te
st_char.c@116@main_/home/james/code/httpd-2.4.7/srclib/apr/tools/gen_test_char.c
@116@@strchr@NULL,!=@-2@12
tools/.libs/gen_test_char.o@/home/james/code/httpd-2.4.7/srclib/apr/tools@gen_te
st_char.c@121@main_/home/james/code/httpd-2.4.7/srclib/apr/tools/gen_test_char.c
@121@@strchr@NULL,!=@-2@12
@
@
"apache2.4.7.out" [converted] 1537899L, 307001379C
```

Talos Demo – Identifying Error Return Values

Talos identifies error return values

Found error return value for status_handler

```
        status_handler_/home/james/code/httpd-2.4.7/modules/generators/mod_statu
s.c has an error return /home/james/code/httpd-2.4.7/modules/generators/mod_stat
us.c:248 84774
```

status_handler function

```
    if (!ap_exists_scoreboard_image()) {
        ap_log_rerror(APLOG_MARK, APLOG_ERR, 0, r, APLOGNO(01237)
                      "Server status unavailable in inetd mode");
        return HTTP_INTERNAL_SERVER_ERROR;
    }

    r->allowed = (AP_METHOD_BIT << M_GET);
    if (r->method_number != M_GET)
        return DECLINED;

    ap_set_content_type(r, "text/html; charset=ISO-8859-1");

    /*
                                                          248,9         25%
```

TALOS DEMO – SYNTHESIZING AND INSERTING SWRR

Talos synthesizes and inserts an SWRR into *status_handler* function

status_handler function

```
static int status_handler(request_rec *r)
{
        return 500;

    const char *loc;
    apr_time_t nowtime;
    apr_interval_time_t up_time;
    int j, i, res, written;
    int ready;
    int busy;
    unsigned long count;
    unsigned long lres, my_lres, conn_lres;
    apr_off_t bytes, my_bytes, conn_bytes;
    apr_off_t bcount, kbcount;
                                                    184,2-9       18%
```

MITIGATION: SUMMARY

- Prevents adversaries to exploit vulnerabilities
  - Disallows the execution of vulnerable code
- Exchanges functionality loss for security
- The challenge is to preserve unobtrusiveness

Mitigation: Strengths & Drawbacks

- Strengths
  - Patch is simple and effective
  - Can be deployed rapidly
- Drawbacks
  - Causes functionality loss

Fix

- Removes vulnerabilities from code
- Preserves program functionality
- Fix correctness is desired particularly for vulnerabilities

STEPS TO PRODUCE A FIX

1. Finding the faulty statement
2. Synthesizing a patch
3. Testing patch correctness (optional)

Two approaches to produce a fix

- Example-based repair
  - Bottom-up, relies on concrete example inputs
- Property-based repair
  - Top-down, uses expert-defined properties

Example-based Repair

- Requires human-labelled example inputs
  - Positive tests – expected program behavior
  - Negative tests – expose the defect

	Positive Tests	Negative Tests
Before the fix	Pass	Fail
After the fix	Pass	Pass

A Faulty Program

```
// returns x-y  if x > y;  0 if x == y; y-x if x < y
1 int distance(int x, int y) {
2    int result;
3    if (x >y)
4        result = x - y;
5    else if (x == y)
6        result = 0;
7    else
8        result = x - y; // should be y - x
9    return result;
```

Input#	Label	x	y	distance (expected)	distance (actual)
1	Positive	2	1	1	1
2	Positive	3	3	0	0
3	Negative	1	4	3	-3
4	Negative	0	5	5	-5

EXAMPLE-BASED: FINDING THE FAULTY STATEMENT

- Statistical fault localization
  - Faulty statement is executed more in negative tests but fewer in positive tests
  - Run the target program to collect execution count of each statement: #passed and #failed

STATISTICAL FAULT LOCALIZATION

1. Compute a *suspiciousness score* for each statement

$$susp(s) = \frac{failed(s)/totalfailed}{passed(s)/totalpassed + failed(s)/totalfailed}$$

2. Rank each statement by its susp. score

Statement	Susp. Score	#failed	#passed
8 result = x -y	1.0	2	0
5 else if (x == y)	0.67	2	1
3 if (x > y)	0.5	2	2
4 result = x - y	0.0	0	1
6 result = 0	0.0	0	1

EXAMPLE-BASED: SYNTHESIZING A PATCH

- Using pre-defined ways
  - Adding a guard, e.g. *if (…) result = x – y;*
  - Modifying RHS of the assignment, e.g. *result = y - x;*
  - *….*
- Learning from correct code
  - Borrowing code from other similar programs

MODIFYING RHS OF AN ASSIGNMENT

1. Replacing the RHS with *f(…)*
   - … can be function parameters and local variables
2. Finding the constraint that *f(…)* needs to satisfy for the given example inputs

$$f(x, y) = \begin{cases} 3, & x==1 \text{ and } y==4 \\ 5, & x==0 \text{ and } y==5 \end{cases}$$

3. Concretizing f(x, y)

CONCRETIZING F(X, Y)

- Constants
  - ~~3 works for input #3 but not input #4~~
  - ~~5 works for input #4 but not input #3~~
- Arithmetic
  - ~~f(x, y) := x + y~~
  - f(x, y) := y – x
- Comparison
- Logic
- ….

LEARNING FROM CORRECT CODE

- Focuses on missing checks for error-triggering inputs
  - E.g. check on input to prevent buffer overflow
- Requires a donor program
  - Performs same functionality
  - Accepts same inputs
  - Contains a check for error-triggering inputs
- Borrows the check from the donor program

Borrowing the Check from The Donor Program

- Can we borrow the check from FEH (donor) and transfer it to CWebP (recipient)?

CWebP Buffer Overflow

```
int ReadJPEG(…) {
….
// overflow error
rgb = malloc(stride * cinfo.height);
….
}
```

FEH Overflow Check

```
char load(…) {
        ….
        if (height>16) {
            // quit
        }
        ….
}
```

CHALLENGES

- How to identify the required check?
- How to transfer the check from the donor to the recipient?
  - The check is implemented in the code of the donor

Identifying the Check

- Using a *seed input* and an *error-triggering input*
  - Seed input passes the check
  - Error-triggering input fails the check
- Running the donor program with both inputs to identify such check
  - Search all checks in the donor program

Checks	Seed Input	Error Input
if (height > 16)	pass	fail
….	….	….

TRANSFERRING THE CHECK

- How to transfer the check to the recipient program?
  1. Lifts the check to an application-independent form
  2. Finds a location in the recipient to insert the check
  3. Translates the check back to program expressions in the recipient
  4. Inserts the check into the recipient

LIFTING THE CHECK

- Uses symbolic execution to map the check to input fields

height > 16 ➔ input.dinfo.output_height > 16

Finding A Candidate Patch Location

- Where can we insert the check in the recipient?
  - Any location in the recipient where the check can be translated
  - Requires testing to verify patch correctness

TRANSLATING THE CHECK

- Uses symbolic execution to map lifted check to recipient program variables

input.dinfo.output_height > 16 ➔ cinfo.height > 16

INSERTING THE CHECK

CWebP Overflow Check

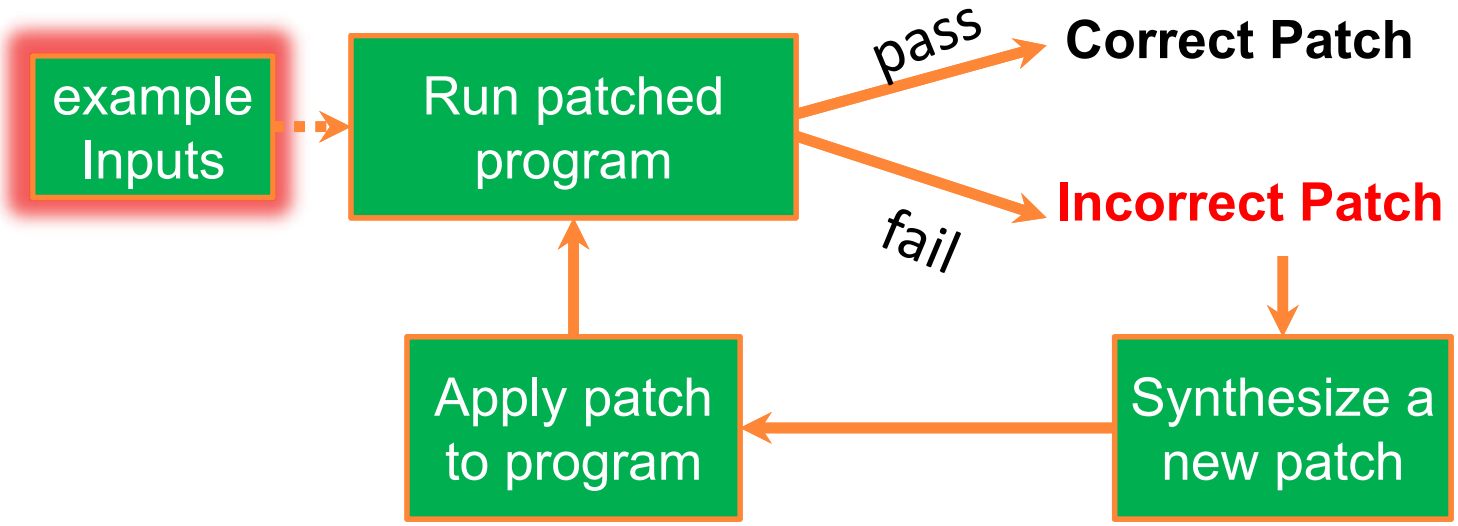
```
int ReadJPEG(…) {
….
// patch
If (cinfo.height > 16) exit(-1);
rgb = malloc(stride * cinfo.height);
….
}
```

FEH Overflow Check

```
char load(…) {
        ….
        if (height>16) {
           // quit
        }
        ….
}
```

EXAMPLE-BASED: TESTING PATCH CORRECTNESS

- Running patched program with example inputs to determine patch correctness



EXAMPLE-BASED REPAIR: SUMMARY

- Relying on example inputs
- Finding the faulty statement
  - Statistical fault localization
- Synthesizing a patch
  - Using pre-defined ways
  - Learning from other programs

Example-Based Repair: Strengths & Drawbacks

- Strengths
  - Generic – (mostly) oblivious to types of vulnerabilities
  - Example inputs can be obtained from test suites
- Drawbacks
  - Less desirable for vulnerabilities – patch correctness is tested using inputs
  - Can take a long time to try out all possible patches

Property-based Repair

- Using expert-defined, program-independent properties to denote a patch
- Patch correctness is enforced by property correctness
  - No need to test patch correctness
  - Does not rely on the completeness of test inputs

USING SAFETY PROPERTIES TO GENERATE VULNERABILITY PATCHES

- A safety property describes the condition when a type of vulnerabilities *cannot* be triggered
  - Abstract: defined in terms of abstract expressions
  - Simple: involving a tiny number of expressions

Safety Property for Buffer Overflow

```
mem_access_upper <= buffer_upper  &&
mem_access_lower >= buffer_lower
```

EXAMPLE VULNERABILITY TYPES

buffer overflow

```
strcpy(buffer, input);
```

bad cast

```
void *p = read_from_file();
struct A *pa = (struct A *)p;
p->field_i = 100;
```

integer overflow

```
short n = strlen(input);
```

Patch Generation

- Input:
  - a target program
  - safety properties defined by experts
  - a test input that triggers the vulnerability
- Output: source code patch

```
if (!safety_property_hold)
    return error;
```

STEPS TO PRODUCE A FIX

1. Finding the faulty statement
2. Synthesizing a patch
3. ~~Testing patch correctness~~

Finding the Faulty Statement

- The statement that violates the safety property
  - Identified during symbolic execution

CHALLENGES TO SYNTHESIZE A PATCH

- How to map a safety property to program expressions, i.e. concretize a safety property?
- Where to place the patch?

CONCRETIZING A SAFETY PROPERTY

- Mapping abstract expressions into program expressions during symbolic execution

Safety Property for Buffer Overflow

```
mem_access_upper <= buffer_upper  &&
mem_access_lower >= buffer_lower
```

Target Program

```
buf = malloc(s);
p = buf;
memcpy(p, q, l)
```

Concretized Safety Property

```
p + l - 1<= buf + s - 1 &&
p >= buf
```

PLACING THE PATCH

- A location before the vulnerability can be triggered
- What if not all expression can be mapped to a same scope?

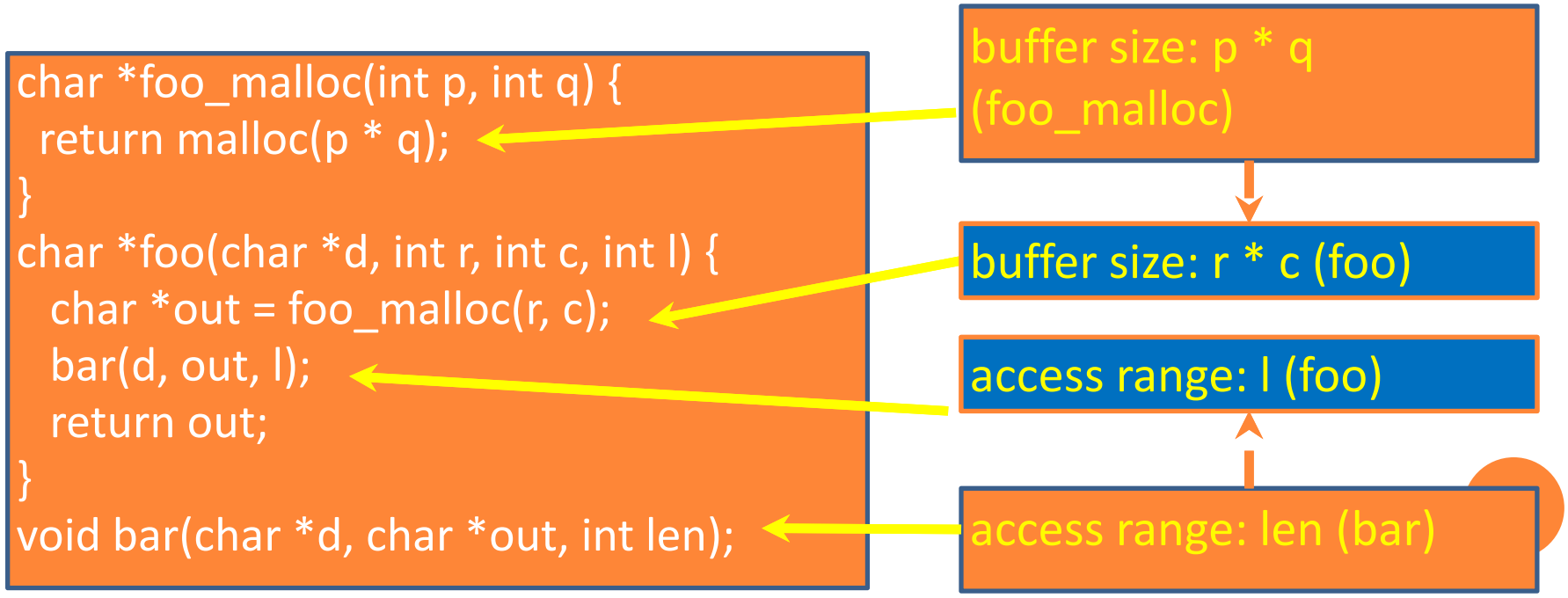
```
char *foo_malloc(int p, int q) {
  return malloc(p * q);
}
char *foo(char *d, int r, int c, int l) {
   char *out = foo_malloc(r, c);
   bar(d, out, l);
   return out;
}
void bar(char *d, char *out, int len);
```

buffer size: p * q (foo_malloc)

access range: len (bar)

Expression Translation

- Translate program expressions across different scopes
  - Based on function summary



SYNTHESIZING THE PATCH

- Target function: foo
- Concretized safety property: r *c >= l
- Error return value: NULL

```
char *foo_malloc(int p, int q) {
  return malloc(p * q);
}
char *foo(char *d, int r, int c, int l) {
   if (!(r * c >= l)) return NULL; // patch
   char *out = foo_malloc(r, c);
   bar(d, out, l);
   return out;
}
void bar(char *d, char *out, int len);
```

PROPERTY-BASED REPAIR: SUMMARY

- Using expert-defined, program-independent properties to generate patches
- Properties need to be mapped to program expressions
- Patch correctness is enforced by property correctness

Property-based Repair: Strengths & Drawbacks

- Strengths
  - Patch correctness is enforced by the correctness of expert-defined properties
  - Properties need to be defined only once
  - More desirable for vulnerabilities
- Drawbacks
  - New properties need to be defined for new vulnerability types
  - Extra Instrumentation may be needed to concretize property

Take Away

- Our goal is to automatically generate patches to repair vulnerabilities
- Mitigation, example-based repair and property-based repair are investigated
- Mitigation is ideal for rapid temporary protection
- For vulnerabilities, property-based repair is more desirable than example-based repair

REFERENCES

- H. D. T. Nguyen, D. Qi, A. Roychoudhury , S. Chandra. *SemFix: Program Repair via Semantic Analysis*. International Conference on Software Engineering 2013.
- S. Sidiroglou-Douskos, E. Lahtinen, F. Long, M. Rinard. *Automatic Error Elimination by Horizontal Code Transfer across Multiple Applications*. ACM SIGPLAN conference on Programming Language Design and Implementation 2015.
- Z. Huang, M. D'Angelo, D. Miyani, D. Lie. *Talos: Neutralizing Vulnerabilities with Security Workaround for Rapid Response.* IEEE Symposium on Security & Privacy 2016.
- Z. Huang, D. Lie, G. Tan, T. Jaeger. *Using Safety Properties to Generate Vulnerability Patches*. IEEE Symposium on Security & Privacy 2019.
- Z. Huang, G. Tan. *Rapidly Mitigating Vulnerabilities with Security Workarounds.* NDSS Workshop on Binary Analysis Research 2019.